华东师范大学数据学院上机实践报告

OS Lab2 彩票调度器实验报告

课程名称： **操作系统**	**年级：** **2023**	**上机实践成绩：**
指导老师：翁楚良	**姓名**：陈子谦	
上机实践名称：彩票调度器实现	**学号**：10235501454	**上机实践日期**：25.3.27

一、实验目的

1. 理解操作系统进程调度算法的基本原理
2. 掌握彩票调度(Lottery Scheduling)算法的实现方法
3. 学习如何在xv6操作系统中添加新的系统调用
4. 掌握在操作系统内核中实现伪随机数生成功能
5. 理解进程创建时状态继承的机制

二、实验内容

实验要求

- 彩票调度的**基本思想**：根据进程持有的彩票数量的比例，为每个运行的进程分配CPU。每个时间片，随机抽签决定彩票的赢家;赢家是在那个时间片上运行的过程。进程拥有的彩票越多，运行的次数就越多。
- **填充pstat结构：** 在内核中填充 `pstat` 结构，存储每个进程的彩票信息的数据结构，并将结果传递到用户空间。 `pstat` 结构不能进行更改，必须按照下面给定的结构使用。

```
struct pstat {
  int inuse[NPROC];   // 是否可运行（1或0）
  int tickets[NPROC]; // 此进程拥有的彩票数
  int pid[NPROC];     // 每个进程的进程ID
  int ticks[NPROC];   // 每个进程已经运行的CPU时间片数量（被选择运行的次数）
};
//NPROC:最大进程数
```

- **新增2个系统调用：**
  - `int settickets(int number)`：设置调用进程的彩票数量。默认情况下，每个进程被分配5个彩票。调用 `settickets()` 使得进程可以增加彩票数量，从而获得更高比例的CPU周期。如果成功，返回0；否则返回-1（例如，如果调用者传入一个小于1的数字）。
  - `int getpinfo(struct pstat *)`：返回有关所有运行进程的信息，包括 `struct pstat` 的4个成员变量信息。可以用此系统调用构建 `struct pstat *ps`。返回0表示成功，返回-1表示失败(例如，如果向内核传递了一个坏指针或NULL指针)。
- **进程继承彩票：** 在创建进程时需要为其分配彩票数。具体来说，确保子进程在创建时继承父进程的彩票数量。如果父进程有10张彩票并调用 `fork()` 创建子进程，子进程也应该获得10张彩票。
- **生成随机数：** 在内核中实现伪随机数生成器，以用于彩票调度的随机彩票抽奖。
- **最终结果**：打印每个进程的信息，包括：进程id（pid），是否可运行（1或0），彩票数（tickets），划分的时间片数量（ticks），以及总时间片数（total ticks）。

三、使用环境

- **操作系统**：xv6-public
- **开发环境**：WSL2 - Ubuntu 24.04
- **硬件环境**：qemu-system-i386

- **编程语言**：C

四、实验过程及结果

1. 创建pstat.h定义数据结构

首先定义存储进程状态信息的数据结构，用于`getpinfo`系统调用返回进程运行状态。

```
#ifndef _PSTAT_H_
#define _PSTAT_H_

#include "param.h"

struct pstat {
  int inuse[NPROC];   // 是否可运行（1或0）
  int tickets[NPROC]; // 此进程拥有的彩票数
  int pid[NPROC];     // 每个进程的进程ID
  int ticks[NPROC];   // 每个进程已运行的CPU时间片数量
};

#endif // _PSTAT_H_
```

2. 实现伪随机数生成器

创建`rand.h`和`rand.c`文件，提供彩票调度所需的随机数生成功能。

rand.h

```
#ifndef _RAND_H_
#define _RAND_H_

// 返回范围在[0, max]内的伪随机数
uint random_at_most(uint max);

// 初始化随机数生成器
void rand_init(void);

#endif // _RAND_H_
```

rand.c

使用线性同余法实现简单的伪随机数生成器，这里参考了一下网上的写法：

```
#include "types.h"
#include "defs.h"
#include "rand.h"

// 线性同余伪随机数生成器参数
#define A 1664525
#define C 1013904223
#define M 0xFFFFFFFF

static uint seed = 1;
```

```c
// 初始化随机数生成器种子
void rand_init(void)
{
  seed = 1;
}

// 生成下一个随机数
static uint next_rand(void)
{
  seed = (A * seed + C) & M;
  return seed;
}

// 返回范围在[0, max]内的伪随机数
uint random_at_most(uint max)
{
  if (max == 0)
    return 0;

  return next_rand() % (max + 1);
}
```

3. 修改进程结构体

在`proc.h`中为`struct proc`添加彩票和时间片统计字段：

```c
struct proc {
  // 原有字段...
  int tickets;                  // 彩票数量
  int ticks;                    // 进程已经运行的时间片数量
};
```

4. 实现系统调用

在系统调用表中添加新的系统调用

syscall.h

修改`syscall.h`添加新的系统调用号：

```c
#define SYS_settickets 22
#define SYS_getpinfo 23
```

syscall.c

更新`syscall.c`注册系统调用：

```
extern int sys_settickets(void);
extern int sys_getpinfo(void);

static int (*syscalls[])(void) = {
  // 原有系统调用...
  [SYS_settickets] sys_settickets,
  [SYS_getpinfo]   sys_getpinfo,
};
```

在sysproc.c中实现系统调用处理函数

```
int
sys_settickets(void)
{
  int tickets;

  if(argint(0, &tickets) < 0)
    return -1;

  return settickets(tickets);
}

int
sys_getpinfo(void)
{
  struct pstat *ps;

  if(argptr(0, (char**)&ps, sizeof(struct pstat)) < 0)
    return -1;

  return getpinfo(ps);
}
```

在user.h中添加系统调用声明

```
int settickets(int);
int getpinfo(struct pstat*);
```

在usys.S中添加汇编代码

```
SYSCALL(settickets)
SYSCALL(getpinfo)
```

5. 实现彩票调度算法

修改`proc.c`中的`allocproc`函数，为新创建的进程分配默认彩票数与初始化时间片：

```c
static struct proc*
allocproc(void)
{
  // 原有代码...
  p->state = EMBRYO;
  p->pid = nextpid++;
  p->tickets = 5;  // 默认分配5个彩票
  p->ticks = 0;    // 初始化时间片为0
  // 原有代码...
}
```

修改`fork`函数，使子进程继承父进程的彩票数：

```c
int
fork(void)
{
  // 原有代码...
  np->tickets = curproc->tickets; // 子进程继承父进程的彩票数
  // 原有代码...
}
```

实现`settickets`和`getpinfo`函数：

```c
int
settickets(int number)
{
  if(number < 1)
    return -1;

  struct proc *p = myproc();

  acquire(&ptable.lock);
  p->tickets = number;
  release(&ptable.lock);

  return 0;
}

int
getpinfo(struct pstat *ps)
{
  if(ps == 0)
    return -1;

  struct proc *p;
  int i;

  acquire(&ptable.lock);

  for(i = 0; i < NPROC; i++) {
    ps->inuse[i] = 0;
    ps->tickets[i] = 0;
    ps->pid[i] = 0;
    ps->ticks[i] = 0;
```

```
  }

  for(p = ptable.proc, i = 0; p < &ptable.proc[NPROC]; p++, i++) {
    ps->inuse[i] = (p->state != UNUSED) ? 1 : 0;
    ps->tickets[i] = p->tickets;
    ps->pid[i] = p->pid;
    ps->ticks[i] = p->ticks;
  }

  release(&ptable.lock);

  return 0;
}
```

重新实现调度器函数，调度器是彩票调度的核心，它按照以下步骤工作：

- 统计总彩票数：遍历进程表，计算所有可运行进程的彩票总和
- 随机抽取彩票：使用我们实现的随机数生成器选择一个中奖号码
- 查找中奖进程：再次遍历进程表，累加每个进程的彩票数，直到找到持有中奖彩票的进程
- 调度运行：增加中奖进程的时间片计数，然后切换到该进程执行

```
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;

  // 初始化随机数生成器
  rand_init();

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // 彩票调度算法实现
    acquire(&ptable.lock);

    // 统计总彩票数
    int total_tickets = 0;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        total_tickets += p->tickets;
      }
    }

    // 如果没有可运行的进程或总彩票数为0，直接进入下一轮循环
    if(total_tickets == 0) {
      release(&ptable.lock);
      continue;
    }

    // 生成随机数作为中奖彩票
    int winner = random_at_most(total_tickets - 1);
```

```
    // 遍历进程表，找到中奖的进程
    int counter = 0;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
      if(p->state != RUNNABLE)
        continue;

      counter += p->tickets;
      if(counter > winner)
        break; // 找到中奖进程
    }

    // 如果没有找到中奖进程，继续下一轮循环
    if(p >= &ptable.proc[NPROC]) {
      release(&ptable.lock);
      continue;
    }

    // 增加进程的时间片计数
    p->ticks++;

    // 设置当前进程为中奖进程
    c->proc = p;
    switchuvm(p);
    p->state = RUNNING;

    swtch(&(c->scheduler), p->context);
    switchkvm();

    // 进程结束运行
    c->proc = 0;

    release(&ptable.lock);
  }
}
```

6. 测试程序实现

testTicket.c

用于设置进程的彩票数并保持运行：

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2) {
    printf(1, "Usage: testTicket <number>\n");
    exit();
  }

  int tickets = atoi(argv[1]);
```

```
  printf(1, "Process %d: Setting tickets to %d\n", getpid(), tickets);

  if(settickets(tickets) < 0) {
    printf(1, "Error: settickets(%d) failed\n", tickets);
    exit();
  }

  int i = 0;
  while(1) {
    i++;
  }
  exit();
}
```

testProcInfo.c

用于获取和显示所有进程的运行状态：

```
#include "types.h"
#include "stat.h"
#include "user.h"
#include "pstat.h"

int
main(void)
{
  struct pstat ps;
  int i;

  if(getpinfo(&ps) < 0) {
    printf(1, "Error: getpinfo() failed\n");
    exit();
  }

  int total_ticks = 0;

  printf(1, "------------ Initially assigned Tickets = 5 ----------\n");
  printf(1, "ProcessID Runnable(0/1) Tickets Ticks\n");

  // 计算总时间片
  for(i = 0; i < NPROC; i++) {
    if(ps.inuse[i]) {
      total_ticks += ps.ticks[i];
    }
  }

  // 打印进程信息
  for(i = 0; i < NPROC; i++) {
    if(ps.inuse[i]) {
      printf(1, "%d %d %d %d\n",
        ps.pid[i],
        ps.inuse[i],
        ps.tickets[i],
        ps.ticks[i]);
    }
```

```
  }

  printf(1, "------------------ Total Ticks = %d -----------------\n",
total_ticks);

  exit();
}
```

7. 修改Makefile

在Makefile中添加新文件并更新编译规则：

```
OBJS = \
    // 原有文件...
    rand.o\
    // 原有文件...

UPROGS=\
    // 原有程序...
    _testTicket\
    _testProcInfo\
```

五、遇到的困难及解决方法

在实验过程中遇到了几个关键问题：

编译器数组边界检查警告：

xv6中的mp.c文件存在数组边界访问问题，导致编译报错。解决方法是在mp.c文件开头添加编译器指令`#pragma GCC diagnostic ignored "-Warray-bounds"`禁用相关警告。

链接器找不到vectors符号：

编译时出现`undefined reference to 'vectors'`错误。问题根源在于，在Makefile中，要先编译trap.c，再生成vector.s，但trap.c需要调用vector.s，两者产生矛盾。可以修改Makefile的OBJS顺序，将vectors放在trap系列之前，即可编译通过。（ps：为什么GitHub直接抓的系统会有这个问题。。不过在解决了问题3后这里的顺序调换回来又能正常编译。。）

sign.pl脚本丢失 && no bootable device：

构建可引导映像时出现`make: ./sign.pl: No such file or directory`错误。这个脚本负责创建可引导的`bootblock`。这个编译的问题简直困扰整个实验，GitHub的源码第一次编译的时候是不会报sign.pl的错误的，但是`make qemu-nox`后会报`no bootable device`的错误，这其实就是因为sign.pl的签名未能正确编译导致。网上关于这个bug的资料少之又少，有的方法说是重新下载源码，有的是给sign.pl加上权限。但都无法解决问题，最终是在Makefile中的bootblock模块，使用perl直接编译sign.pl解决了这个bug

```
bootblock: bootasm.S bootmain.c
    $(CC) $(CFLAGS) -fno-pic -O -nostdinc -I. -c bootmain.c
    $(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c bootasm.S
    $(LD) $(LDFLAGS) -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
    $(OBJDUMP) -S bootblock.o > bootblock.asm
    $(OBJCOPY) -S -O binary -j .text bootblock.o bootblock
    perl ./sign.pl bootblock
```

测试问题：

最初testTicket程序执行太快，无法观察到彩票调度效果。将testTicket.c中的while循环修改为无限循环确保进程持续运行，解决了这个问题。

六、实验结果与分析

输入测试：

```
$make qemu-nox (运行qemu模拟器)
$testTicket 10&; testTicket 15&; testTicket 20&; testTicket 25&; testTicket 30&;
(设置进程的彩票数)
$testProcInfo (输出彩票调度后的进程状态
```

然后运行`testProcInfo`查看进程状态：

```
init: starting sh
$ testTicket 10&; testTicket 15&; testTicket 20&; testTicket 25&; testTicket 30&;
Process 5: Setting tickets to 10
Process 13: Setting tickets to 30
Process 9: Setting tickets to 20
Process 7: Setting tickets to 15
Process 11: Setting tickets to 25
$ testProcInfo
------------ Initially assigned Tickets = 5 ----------
ProcessID Runnable(0/1) Tickets Ticks
1 1 5 13
2 1 5 22
14 1 5 7
5 1 10 106
7 1 15 128
9 1 20 187
11 1 25 206
13 1 30 232
------------------- Total Ticks = 901 -----------------
$ 
```

从结果可以看出，彩票数量越多的进程获得的CPU时间（ticks)也越多，这符合彩票调度的设计初衷。例如，彩票数为30的进程获得了232个时间片，而彩票数为10的进程只获得了106个时间片，大致符合3:1的比例关系。